

UNIVERSIDAD ARGENTINA DE LA EMPRESA | CARRERAS DE
INFORMÁTICA

TP 06 · Paradigma Orientado a Objetos · TPO

Traducción del SDK Unitree: De Python a una Librería Java

Los alumnos reciben el SDK oficial en Python y lo traducen a una librería orientada a objetos en Java

DURACIÓN TALLER

2 horas

MODALIDAD

TPO — Obligatorio

ROBOT

G1 y Go2

TRABAJO

Individual o parejas

NIVEL

Avanzado

1. Título del Trabajo Práctico

Traducción del SDK Unitree: De Python a una Librería Java

2. Materia

Paradigma Orientado a Objetos — Carreras de Informática, UADE

3. Duración Estimada

Taller presencial: 2 horas (120 minutos). Incluye repaso de principios POO, análisis del SDK real, diseño del diagrama UML en pizarrón, implementación guiada y presentación.

Extensión evaluable: Entrega formal de la librería completa con todas las clases implementadas, diagrama UML en herramienta formal (draw.io, PlantUML o Lucidchart), suite de pruebas unitarias con

JUnit 5, y documentación con Javadoc. Duración estimada: 2 semanas.

4. Fundamentación

El Paradigma Orientado a Objetos (POO) es el paradigma dominante en el desarrollo de software moderno. Sus pilares —abstracción, encapsulamiento, herencia y polimorfismo— son herramientas de diseño que permiten construir sistemas complejos de manera modular, mantenible y escalable. Sin embargo, estos conceptos suelen enseñarse con ejemplos desconectados de la práctica real: animales que hablan, figuras geométricas que calculan su área.

En este TP, el alumno no parte de cero: recibe el SDK del robot Unitree ya escrito en Python y debe *traducirlo* a Java, construyendo una librería orientada a objetos en el proceso. Esa traducción obliga a leer el código fuente Python con atención y a tomar decisiones genuinas de diseño: ¿qué atributos deben ser privados? ¿Por qué `Robot` debe ser abstracta? ¿Qué comportamiento comparten el G1 y el Go2 y cuál es específico de cada uno? ¿Por qué separar `ValidadorSeguridad` del robot en lugar de incluir la validación en el método `ejecutarComando()`?

El ejercicio agrega además una dimensión de traducción entre lenguajes: el alumno debe respetar las convenciones Java (camelCase, tipado estático, `abstract class`, visibilidad explícita) mientras preserva la lógica del SDK fuente. Esta traducción fuerza una comprensión profunda del código, no solo su copia mecánica.

Estas preguntas no tienen respuesta única, y discutir las es el núcleo de la actividad: el alumno aprende POO tomando decisiones de diseño reales y defendiéndolas.

5. Objetivos Generales

- Aplicar los cuatro pilares del POO (abstracción, encapsulamiento, herencia y polimorfismo) en el diseño de una librería de software real.
- Diseñar una jerarquía de clases coherente que represente los conceptos del dominio robótico: robots, comandos, rutinas, sensores, sesiones y seguridad.
- Traducir el SDK Python provisto por la cátedra a una librería Java, respetando las convenciones del lenguaje (camelCase, tipado estático, Javadoc).
- Documentar las decisiones de diseño a través de un diagrama UML y un informe técnico.

6. Objetivos Específicos

- Implementar la clase abstracta `Robot` con los métodos abstractos `conectar()`, `getSensores()` y `ejecutarInterno()`.
- Implementar las clases concretas `UnitreeGo2` y `UnitreeG1` extendiendo `Robot` y especializando los métodos abstractos.
- Implementar las clases `Comando`, `Rutina`, `Sensor`, `Sesion`, `Logger` y `ValidadorSeguridad` con responsabilidades claramente delimitadas.
- Demostrar polimorfismo: escribir una función que opere sobre cualquier instancia de `Robot` sin conocer su tipo concreto.
- Generar un diagrama UML con las relaciones (herencia, composición, uso) entre todas las clases.

7. Contenidos de la Materia que se Trabajan

- **Conceptos básicos del paradigma orientado a objetos:** objetos, clases, mensajes, identidad, estado y comportamiento; el robot como objeto con estado interno y comportamiento público.
- **Tipos de relaciones:** herencia (`UnitreeGo2` *es-un* `Robot`), composición (`Robot` *tiene-un* `Logger`), agregación (`Rutina` *tiene* `Comando` s), dependencia (`Sesion` *usa* `ValidadorSeguridad`).
- **Herramientas de modelado:** StarUML, draw.io o Lucidchart para crear y mantener diagramas UML del sistema.
- **Lenguaje de Modelado UML:** notación estándar para clases, relaciones, visibilidad (+/-/#), multiplicidad y estereotipos (`<<abstract>>`).
- **Diagramas de diseño:** diagrama de clases completo de la librería con todas las clases, atributos, métodos y relaciones; trazabilidad directa al código.
- **Patrones para la asignación de responsabilidad (GRASP):** Experto en Información (quien conoce los datos del robot, responde), Creador (quién instancia `Sesion` y `Rutina`), Controlador (la `Sesion` como fachada de operaciones), Bajo Acoplamiento, Alta Cohesión.
- **Correspondencia entre modelado y código:** cada clase del diagrama UML se implementa directamente en el código; los atributos y métodos del diagrama aparecen en el código sin divergencias.

Abstracción, encapsulamiento, herencia y polimorfismo: pilares aplicados en la jerarquía `Robot` → `UnitreeGo2` / `UnitreeG1` ; Template Method en `ejecutarComando()` como ejemplo de polimorfismo estructurado.

8. Descripción General de la Actividad

Los alumnos recibirán el SDK del robot Unitree escrito en Python y lo **traducirán a Java**, construyendo una librería orientada a objetos que oculta los detalles de comunicación de bajo nivel y expone una API de alto nivel. La librería Java resultante debe ser utilizable por cualquier otro TP de la propuesta: en teoría, el código del TP01, TP02 o TP03 podría reescribirse en Java usando esta librería sin conocer la implementación interna.

La actividad comienza con la lectura del SDK en Python (provisto por la cátedra) para identificar qué abstracciones son necesarias y cómo deben mapearse al sistema de tipos de Java. Luego, el alumno diseña el diagrama UML en papel o pizarrón, lo refina con el docente y comienza la implementación en Java. Al finalizar, demuestra polimorfismo ejecutando una misma función con instancias de `UnitreeGo2` y `UnitreeG1`.

9. Modalidad de Trabajo

Organización: Individual o en parejas. Se recomienda trabajo en pareja para el diseño UML (es más productivo con dos perspectivas) e individual para la implementación.

- **Simulado / código puro:** Toda la actividad transcurre en el editor de código. No se requiere robot físico.
- **Con SDK simulado:** Las clases implementadas se integran con el módulo `unitree_sdk_sim` de la cátedra para verificar que la abstracción funciona correctamente con el simulador.
- **Demostración con robot real:** La librería completa se instancia con las clases reales del SDK de Unitree y se demuestra sobre el robot físico bajo supervisión.

10. Recursos Necesarios

Software

- Java 17+ con `javac` y un IDE (IntelliJ IDEA, Eclipse, o VS Code + Extension Pack for Java).
- Python 3.10+ para leer y analizar el SDK fuente — *no se implementa en Python*.
- Herramienta UML: draw.io (web, gratuita), PlantUML, o papel/pizarrón para el taller.
- `JUnit 5` para pruebas unitarias (extensión evaluable).

Archivos provistos por la cátedra

- `unitree_sdk_sim.py` : SDK fuente en Python — se lee para comprender la lógica antes de traducir a Java.
- `tp06_sdk_real_simplificado.py` : versión simplificada del SDK real en Python para estudiar y analizar.
- `tp06_plantilla.java` : estructura de clases Java vacías con comentarios Javadoc guía para la traducción.

11. Desarrollo del Taller de 2 Horas

Tiempo	Actividad del Docente	Actividad de los Alumnos
00–15 min	Repaso de herencia, polimorfismo y clases abstractas con un ejemplo concreto. Presenta el SDK real simplificado en Python y señala sus limitaciones de diseño (acoplamiento, sin validación, sin logging) y las diferencias con Java.	Leen el SDK fuente en Python. Identifican qué está mal diseñado y planifican cómo traducirlo a Java respetando las convenciones del lenguaje.
15–35 min	Conduce el diseño colaborativo del diagrama UML en pizarrón: ¿cuáles son las clases? ¿Qué relaciones hay entre ellas? ¿Qué es abstracto? ¿Qué es composición?	Proponen clases y relaciones, discuten entre sí y con el docente. Bosquejan su propio diagrama UML en papel.
35–80 min	Guía la implementación de las clases principales: <code>Robot</code> (abstracta), <code>UnitreeGo2</code> , <code>UnitreeG1</code> , <code>ValidadorSeguridad</code> , <code>Logger</code> . Hace preguntas socráticas: "¿por qué ese método debería ser privado?"	Implementan las clases principales. Verifican herencia y encapsulamiento ejecutando código de prueba básico.
80–100 min	Propone el desafío de polimorfismo: escribir un método <code>ejecutarRutinaEnRobot(Robot robot, Rutina rutina)</code> que funcione con cualquier subclase de <code>Robot</code> .	Implementan el método polimórfico en Java. Prueban instanciando tanto <code>UnitreeGo2</code> como <code>UnitreeG1</code> y verifican que el comportamiento es correcto en ambos casos.
100–115 min	Recorre los puestos para verificar implementación de <code>Sesion</code> y <code>Logger</code> . Señala casos donde la responsabilidad de una clase no está claramente delimitada.	Implementan <code>Sesion</code> y <code>Logger</code> . Refinan el diagrama UML para reflejar la implementación real.
115–120 min	Cierre: ¿qué clase fue más difícil de diseñar y por qué? Síntesis de los principios POO aplicados.	Presentan brevemente su diagrama UML final y la función polimórfica.

12. Consigna para los Alumnos

Trabajo Práctico N° 6 — Paradigma Orientado a Objetos

Recibirán el SDK del robot Unitree escrito en Python y lo **traducirán a Java**. La librería Java debe incluir las siguientes clases:

- `Robot` : clase abstracta base con `conectar()` , `ejecutarComando()` , `getSensores()` , `desconectar()` .
- `UnitreeGo2` y `UnitreeG1` : subclases concretas con comportamiento especializado.
- `Comando` : representa un comando con tipo, parámetros y timestamp.
- `Rutina` : secuencia ordenada de comandos con nombre y descripción.
- `Sensor` : abstracción de sensores (batería, temperatura, LiDAR).
- `Sesion` : registra el inicio, fin y comandos ejecutados durante una sesión.
- `Logger` : registra eventos con nivel (INFO, WARN, ERROR) y timestamp.
- `ValidadorSeguridad` : valida que un comando es seguro antes de ejecutarlo.

Deben demostrar: encapsulamiento (atributos privados con getters), herencia (`Go2` y `G1` extienden `Robot`), polimorfismo (método que opera sobre cualquier `Robot`) y abstracción (`Robot` abstracta). El código debe estar en **Java** y compilar con `javac` .

Entregables: código fuente Java + diagrama UML + informe técnico.

13. Desarrollo Técnico Esperado

La clase abstracta `Robot` define el contrato que deben cumplir todos los robots: conectar, ejecutar comandos (con validación), obtener datos de sensores y desconectar. El método `ejecutarComando()` implementa el patrón Template Method: llama a `ValidadorSeguridad` antes de llamar al método interno `ejecutarInterno()` , que cada subclase especializa según su SDK particular. Este método se declara `final` en Java para garantizar que ninguna subclase pueda saltar la validación.

`ValidadorSeguridad` es una clase independiente (no una función del robot) porque su lógica puede evolucionar, puede ser intercambiada, y puede ser probada en forma aislada. Esta separación ilustra el principio de responsabilidad única y el beneficio de la composición sobre la herencia.

`Sesion` y `Logger` colaboran: la sesión registra qué se hizo, el logger registra cómo fue. Ambas son inyectadas en el robot como dependencias, no creadas internamente, lo que facilita las pruebas unitarias.

14. Ejemplo de Estructura de Código (Java)

Punto de partida: el SDK fuente está en Python (`tp06_sdk_real_simplificado.py`). El objetivo es traducir su lógica a Java — no copiarlo, sino reescribirlo respetando las convenciones del lenguaje: camelCase, tipado estático, `abstract class` , `final` , Javadoc.

```
// Libreria_Unitree.java - Traducción del SDK Unitree (Python) → Java
// Cada clase reproduce la lógica del módulo Python aplicando convenciones Java
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.*;

// — Comando —
public class Comando {
    private final String tipo;
    private final Map<String, Object> parametros;
    private final LocalDateTime timestamp;

    public Comando(String tipo, Map<String, Object> parametros) {
        this.tipo = tipo;
        this.parametros = parametros != null ? parametros : new HashMap<>();
        this.timestamp = LocalDateTime.now();
    }

    public String getTipo() { return tipo; }
    public Map<String, Object> getParametros() { return Collections.unmodifiableMap(parametros); }
    @Override public String toString() { return tipo + "(" + parametros + ")"; }
}

// — Rutina —
public class Rutina {
    private final String nombre;
    private final List<Comando> comandos = new ArrayList<>();

    public Rutina(String nombre) { this.nombre = nombre; }

    public void agregar(Comando c) { comandos.add(c); }
    public List<Comando> getComandos() { return Collections.unmodifiableList(comandos); }
    public int size() { return comandos.size(); }
    public String getNombre() { return nombre; }
}

// — Logger —
public class Logger {
    private static final DateTimeFormatter FMT =
        DateTimeFormatter.ofPattern("HH:mm:ss");
    private final String prefijo;
    private final List<String> registros = new ArrayList<>();

    public Logger(String prefijo) { this.prefijo = prefijo; }

    public void info (String msg) { log("INFO", msg); }
    public void warn (String msg) { log("WARN", msg); }
```

```

    public void error(String msg) { log("ERROR", msg); }

    void log(String nivel, String msg) {
        String entrada = String.format("[%s] [%s] %s: %s",
            LocalDateTime.now().format(FMT), prefijo, nivel, msg);
        registros.add(entrada);
        System.out.println(entrada);
    }
}

// — ValidadorSeguridad —————
public class ValidadorSeguridad {
    private static final double VEL_MAX = 0.5;
    private static final double ANG_MAX = 180.0;
    private static final List<String> PROHIBIDOS =
        List.of("SALTO", "CAIDA_LIBRE", "GIRO_RAPIDO");

    public record Resultado(boolean seguro, String motivo) {}

    public Resultado esSeguro(Comando cmd) {
        if (PROHIBIDOS.contains(cmd.getTipo()))
            return new Resultado(false, "Tipo prohibido: " + cmd.getTipo());
        double vel = (double) cmd.getParametros().getOrDefault("velocidad", 0.0);
        if (vel > VEL_MAX)
            return new Resultado(false,
                String.format("Velocidad %.1f m/s excede límite %.1f m/s", vel, VEL_MAX));
        double ang = Math.abs((double) cmd.getParametros().getOrDefault("angulo", 0.0));
        if (ang > ANG_MAX)
            return new Resultado(false,
                String.format("Ángulo %.0f° excede límite %.0f°", ang, ANG_MAX));
        return new Resultado(true, "OK");
    }
}

// — Robot (clase abstracta) —————
public abstract class Robot {
    protected final String modelo;
    protected final String ip;
    protected boolean conectado = false;
    protected int bateria = 0;
    private final Logger logger;
    private final ValidadorSeguridad validador;

    public Robot(String modelo, String ip) {
        this.modelo = modelo;
        this.ip = ip;
        this.logger = new Logger(modelo);
        this.validador = new ValidadorSeguridad();
    }
}

```

```

    }

    // — Contrato abstracto (equivale a @abstractmethod de Python) —
    public abstract boolean          conectar();
    public abstract Map<String, Object> getSensores();
    protected abstract boolean      ejecutarInterno(Comando cmd);
    public abstract void             desconectar();

    /** Template Method: valida → ejecuta → registra (final: no se puede saltar) */
    public final boolean ejecutarComando(Comando cmd) {
        var r = validador.esSeguro(cmd);
        if (!r.seguro()) { logger.warn("Rechazado: " + r.motivo()); return false; }
        boolean ok = ejecutarInterno(cmd);
        if (ok) logger.info ("Comando " + cmd + " → OK");
        else    logger.error("Comando " + cmd + " → FALLO");
        return ok;
    }

    public int ejecutarRutina(Rutina rutina) {
        int exitos = 0;
        for (Comando c : rutina.getComandos())
            if (ejecutarComando(c)) exitos++;
        logger.info("Rutina '" + rutina.getNombre() + "': "
            + exitos + "/" + rutina.size() + " OK");
        return exitos;
    }

    public String  getModelo()    { return modelo; }
    public boolean isConnected() { return conectado; }
    public int     getBateria()   { return bateria; }
    protected Logger getLogger() { return logger; }
}

// — UnitreeGo2 —
public class UnitreeGo2 extends Robot {
    public static final int PATAS = 4;

    public UnitreeGo2(String ip) { super("Go2", ip); }

    @Override public boolean conectar() {
        conectado = true; bateria = 95;
        getLogger().info("Go2 conectado en " + ip); return true;
    }

    @Override public Map<String, Object> getSensores() {
        return Map.of("patas", PATAS, "bateria", bateria, "modelo", modelo);
    }

    @Override protected boolean ejecutarInterno(Comando cmd) {
        return true; // llamada real al SDK Go2 iría aquí
    }
}

```

```

    }
    @Override public void desconectar() {
        conectado = false; getLogger().info("Go2 desconectado.");
    }
}

// — UnitreeG1 —————
public class UnitreeG1 extends Robot {
    public static final int ALTURA_CM = 127;

    public UnitreeG1(String ip) { super("G1", ip); }

    @Override public boolean conectar() {
        conectado = true; bateria = 88;
        getLogger().info("G1 conectado en " + ip); return true;
    }
    @Override public Map<String, Object> getSensores() {
        return Map.of("altura_cm", ALTURA_CM, "bateria", bateria, "modelo", modelo);
    }
    @Override protected boolean ejecutarInterno(Comando cmd) {
        return true; // llamada real al SDK G1 iría aquí
    }
    @Override public void desconectar() {
        conectado = false; getLogger().info("G1 desconectado.");
    }
}

// — Demostración de POLIMORFISMO —————
public class Main {
    /** Funciona con cualquier subclase de Robot sin modificar una sola línea */
    static void demoPolimorfismo(Robot robot) {
        robot.conectar();
        System.out.println("  Sensores: " + robot.getSensores());

        Rutina rutina = new Rutina("Saludo básico");
        rutina.agregar(new Comando("POSE", Map.of("nombre", "saludo")));
        rutina.agregar(new Comando("MOVER", Map.of("velocidad", 0.3, "duracion", 2)));
        rutina.agregar(new Comando("GIRAR", Map.of("angulo", 90.0)));
        rutina.agregar(new Comando("MOVER", Map.of("velocidad", 0.0, "duracion", 0)));

        robot.ejecutarRutina(rutina);
        robot.desconectar();
    }

    public static void main(String[] args) {
        System.out.println("=== Go2 ===");
        demoPolimorfismo(new UnitreeGo2("192.168.1.10"));
        System.out.println("\n=== G1 ===");
    }
}

```

```
demoPolimorfismo(new UnitreeG1("192.168.1.20"));  
}  
}
```

15. Entregables

- **Código fuente Java** (carpeta del proyecto con los archivos `.java`): implementación completa de todas las clases, compilable con `javac`.
- **Diagrama UML** (imagen PNG/PDF o link a draw.io): jerarquía de clases con relaciones de herencia, composición y uso. Debe incluir atributos y métodos clave con visibilidad (`+` público, `-` privado, `#` protegido).
- **Demo de polimorfismo**: captura de consola mostrando la misma función operando sobre `UnitreeGo2` y `UnitreeG1`.
- **Informe técnico (1–2 páginas)**: justificación de las decisiones de diseño más importantes: por qué `Robot` es abstracta, por qué `ValidadorSeguridad` está separada, qué se gana con el patrón Template Method en `ejecutar_comando()`.

16. Criterios de Evaluación

- La clase `Robot` es efectivamente abstracta y no puede instanciarse directamente.
- Las subclases sobrescriben correctamente los métodos abstractos con comportamiento especializado.
- Los atributos sensibles están encapsulados (privados o protegidos) con acceso controlado.
- El método polimórfico opera correctamente con ambas subclases sin usar `instanceof`.
- El diagrama UML es correcto, completo y refleja la implementación real.
- El informe justifica las decisiones de diseño con argumentos de POO, no solo describe el código.

17. Rúbrica de Evaluación

Criterio	Indicadores de logro	Puntaje máximo
Diagrama UML	Incluye todas las clases, las relaciones (herencia, composición) son correctas, los atributos y métodos tienen visibilidad correcta.	25
Implementación de clases	Todas las clases están implementadas. La abstracción, herencia y encapsulamiento están correctamente aplicados.	30
ValidadorSeguridad y Logger	Validan correctamente comandos peligrosos. El logger registra todos los eventos con nivel y timestamp. Están desacoplados del robot.	20
Polimorfismo demostrado	El método demo funciona con ambas subclases. No usa <code>instanceof</code> para diferenciar tipos.	15
Informe técnico	Justifica las decisiones de diseño con vocabulario POO. No se limita a describir el código.	10
TOTAL		100

18. Medidas de Seguridad

Modalidad código puro (sin robot físico): No hay riesgos físicos. La librería corre íntegramente en la PC.

Si la librería se conecta al robot físico:

- El método `ejecutar_comando()` debe pasar siempre por `ValidadorSeguridad`. No se puede bypassar.
- Las velocidades de movimiento deben estar limitadas a 0,5 m/s en el validador.
- La clase `UnitreeG1` debe implementar una verificación adicional de equilibrio antes de ejecutar comandos de movimiento.

- La sesión debe registrar todos los comandos ejecutados para auditoría post-sesión.

19. Posibles Extensiones del TP

- **Extensión 1 — Pruebas unitarias:** Suite de tests con `JUnit 5` que verifique el comportamiento de `ValidadorSeguridad`, `Logger` y el polimorfismo.
- **Extensión 2 — Patrón Observer:** `Sesion` notifica a observadores (logger, dashboard) cada vez que se ejecuta un comando.
- **Extensión 3 — Patrón Factory:** Una clase `RobotFactory` que instancia el robot correcto a partir de un string de configuración.
- **Extensión 4 — Serialización:** Las clases `Sesion` y `Rutina` pueden serializarse a JSON y recargarse en otra ejecución.
- **Extensión 5 — Integración con TPs anteriores:** Reescribir el código del TP01, TP02 o TP03 en Java usando esta librería como base.

20. Resultado Esperado

Al finalizar el taller, el alumno contará con una librería Java funcional que traduce el SDK del robot Unitree (originalmente en Python) a una jerarquía de clases bien diseñada, un diagrama UML que documenta esa jerarquía y una demostración de polimorfismo. Más importante, habrá tomado y justificado decisiones de diseño reales en un contexto de software genuino, y habrá experimentado de primera mano cómo los mismos conceptos POO se expresan en lenguajes distintos.

21. Cierre y Reflexión para los Alumnos

Hoy no programaron un robot: diseñaron una librería que alguien más podría usar para programar un robot. Esa diferencia es fundamental en ingeniería de software.

Cuando diseñaron `Robot` como clase abstracta, estaban diciendo: "no importa qué robot uses, todos tienen que poder conectarse, recibir comandos y leer sensores". Eso es una API. Cuando separaron `ValidadorSeguridad` del robot, estaban diciendo: "la responsabilidad de validar no le pertenece al robot, le pertenece a alguien externo que puede cambiar sin afectar al robot". Eso es el principio de responsabilidad única.

Estas no son ideas académicas: son las mismas decisiones que toman los equipos de ingeniería cuando diseñan SDKs, frameworks y librerías de producción. Hoy las tomaron ustedes, con un sistema real como contexto.